

THE MATCOM MINIX

Informe Técnico

Miguel Zamora Grupo: C-212

Alfonso Tejeda Rodríguez Grupo: C-212

14 de mayo de 2026

Contents

1	Introducción	2
2	Desarrollo y resultados por componente	2
2.1	Instalación y configuración de VMware y MINIX	2
2.2	Personalización del mensaje de bienvenida	3
2.3	Depuración de un bug en <code>pthread</code>	3
2.4	Implementación del comando <code>tree</code>	5
2.5	Penalización por uso intensivo de CPU	7
3	Resultados globales y discusión	14
4	Conclusiones	14
5	Referencias consultadas	15
6	Declaración de uso de IA	15

1 Introducción

El presente proyecto se desarrolló en el marco de la asignatura Sistemas Operativos, con el objetivo de abordar de manera práctica y progresiva los principios fundamentales que rigen el funcionamiento de un sistema operativo real. Los principios estudiados abarcan la gestión de procesos, la planificación del procesador, el manejo de concurrencia mediante hilos y la interacción con el sistema de archivos mediante llamadas al sistema. Como plataforma de trabajo se utilizó MINIX 3, un sistema operativo de micronúcleo de uso académico diseñado por Andrew S. Tanenbaum, cuya arquitectura modular permite intervenir componentes internos con relativa accesibilidad sin comprometer la estabilidad de un entorno de producción.

La metodología adoptada exigió intervenir código productivo real del sistema: localizar archivos fuente relevantes, comprender su funcionamiento, modificar la lógica existente, recompilar e instalar los cambios dentro de la máquina virtual, y validar los resultados experimentalmente. Este enfoque contrastó con ejercicios aislados o simulaciones, ya que cada modificación tuvo efecto directo sobre el comportamiento observable del sistema operativo en ejecución.

Los objetivos específicos del proyecto fueron cuatro: (1) preparar el entorno de trabajo en MINIX sobre VMware y realizar una primera modificación visible del sistema mediante la personalización de `/etc/motd`; (2) depurar un bug real en la capa de compatibilidad `pthread_*`, que provocaba bloqueo indefinido en la segunda llamada a `pthread_mutex_trylock`; (3) implementar desde cero el comando de usuario `tree`, que imprime recursivamente la estructura de directorios de forma similar al comando homónimo en Linux; y (4) extender el planificador de espacio de usuario (`sched`) para detectar procesos *CPU-bound* y aplicarles una penalización gradual de prioridad dentro de ventanas temporales de planificación.

2 Desarrollo y resultados por componente

En esta sección se documenta, para cada uno de los puntos requeridos, el proceso completo de trabajo: desde el análisis inicial y las decisiones de diseño, pasando por los cambios concretos realizados en el código o la configuración del sistema, hasta la verificación experimental de su correcto funcionamiento.

2.1 Instalación y configuración de VMware y MINIX

El entorno de trabajo se preparó instalando VMware Workstation sobre el sistema anfitrión (CachyOS, distribución basada en Arch Linux), y creando una máquina virtual para ejecutar MINIX 3.3.0, la última versión estable disponible en <https://www.minix3.org>. La imagen ISO utilizada fue `minix_R3.3.0-588a35b.iso`, descargada del repositorio oficial del proyecto.

Los parámetros de configuración de la máquina virtual fueron los siguientes:

- **Memoria RAM:** 1 GB.
- **Núcleos de CPU:** 2. Consistente con el objetivo didáctico; además, usar dos núcleos simplifica la observación de la evolución de prioridades en el scheduler.
- **Disco duro virtual:** 4 GB. Suficiente para alojar el árbol fuente completo y los objetos generados por la compilación.
- **Red:** NAT. Permite acceso a Internet desde la VM para descargar paquetes mediante `pkgin`, sin exponer la VM directamente a la red local.

La instalación de MINIX siguió los pasos de la guía oficial: arranque desde ISO, particionado automático con `autopart` e instalación del conjunto de paquetes base. Tras la instalación base se configuraron las herramientas de desarrollo necesarias:

Listing 1: Instalación de herramientas de desarrollo en MINIX

```
pkgin update
pkgin install git clang binutils
```

El repositorio del proyecto fue clonado dentro de la VM y utilizado como árbol fuente activo. A lo largo del proyecto se adoptó el flujo de trabajo estándar de MINIX: editar fuentes → **make** en el subdirectorio afectado → **make install** → reiniciar el servicio o hacer reboot según correspondiera.

No se presentaron incidencias graves durante la instalación. El único ajuste requerido fue ampliar el tiempo de espera del gestor de arranque (**menu.lst**) para facilitar la selección de kernel durante las sucesivas pruebas de recompilación del scheduler.

2.2 Personalización del mensaje de bienvenida

Ruta modificada: `/etc/motd` (en el repositorio: `etc/motd`).

El archivo `/etc/motd` (*message of the day*) es leído por el proceso de *login* y su contenido se imprime en el terminal al iniciar sesión. No requirió recompilación ni reinicio del sistema: bastó con modificar el archivo de texto plano y los cambios fueron visibles en el siguiente inicio de sesión.

Los permisos del archivo son **-rw-r-r-** (644), propiedad de **root**. La edición se realizó con el editor **vi**, disponible en la instalación base de MINIX:

Listing 2: Edición del mensaje de bienvenida como root

```
vi /etc/motd
```

El contenido final del archivo es:

Listing 3: Contenido de `/etc/motd`

```
*=====*
*      Bienvenido a esto que dice ser un      *
*      Sistema Operativo.                     *
*      Manicomio de Matematica y Computacion  *
*      Colina del Terror de La Habana (UH)    *
*=====*
```

La verificación se realizó cerrando la sesión con **exit** e iniciando sesión nuevamente con el usuario **root**. El mensaje apareció correctamente en el terminal inmediatamente después del prompt de *login*.

No se encontraron restricciones de permisos adicionales: el archivo es editable directamente por **root** sin necesidad de montar el sistema de archivos en modo especial.

2.3 Depuración de un bug en pthread

Contexto

Al ejecutar el programa de prueba provisto en la orientación — que llama a **pthread_mutex_trylock** dos veces consecutivas sobre el mismo mutex — el programa fallaba al ejecutar la segunda llamada. La primera invocación retornaba correctamente con valor 0, pero la segunda desencadenaba una recursión infinita que hacía crecer la pila de llamadas sin límite, impidiendo que **pthread_mutex_unlock** y **pthread_mutex_destroy** se ejecutaran. El proceso debía terminarse manualmente con **Ctrl+C**.

Análisis

En MINIX 3, la implementación real de hilos reside en `libmthread`. Las funciones con prefijo `pthread_*` son una capa de compatibilidad delegada definida en `minix/lib/libmthread/pthread_compat.c`, que en condiciones normales simplemente delega en la función equivalente `mthread_*`.

Al inspeccionar el archivo `pthread_compat.c` se localizó la función `pthread_mutex_trylock`:

Listing 4: Código erróneo original — llamada recursiva a sí misma

```
1 int pthread_mutex_trylock(pthread_mutex_t *mutex)
2 {
3     if (PTHREAD_MUTEX_INITIALIZER == *mutex) {
4         mthread_mutex_init(mutex, NULL);
5     }
6
7     return pthread_mutex_trylock(mutex); /* BUG: recursion infinita */
8 }
```

La causa raíz es una **llamada recursiva directa**: la línea de retorno invoca a la propia función en lugar de delegar en `mthread_mutex_trylock`. El bloque de inicialización con `PTHREAD_MUTEX_INITIALIZER` es correcto y sigue el patrón de las demás funciones del archivo; el error está exclusivamente en la última línea. Al ejecutarse, la llamada a `pthread_mutex_trylock` entra en recursión infinita sin llegar jamás a `mthread`: la pila crece sin límite hasta que el proceso queda en un estado irrecuperable (cuelgue observable). El patrón del archivo establece que cada `pthread_X` debe delegar en `mthread_X`; ésta era la única función que referenciaba su propio nombre en la línea de retorno.

Corrección

La corrección mínima consiste en reemplazar la llamada recursiva por la delegación correcta a `mthread_mutex_trylock`:

Listing 5: Diff funcional de la corrección en `pthread_compat.c`

```
--- a/minix/lib/libmthread/pthread_compat.c
+++ b/minix/lib/libmthread/pthread_compat.c
@@ int pthread_mutex_trylock(pthread_mutex_t *mutex)
 {
     if (PTHREAD_MUTEX_INITIALIZER == *mutex) {
         mthread_mutex_init(mutex, NULL);
     }
-    return pthread_mutex_trylock(mutex);
+    return mthread_mutex_trylock(mutex);
 }
```

El cambio es de una sola línea: reemplazar la llamada recursiva `pthread_mutex_trylock` por la delegación correcta `mthread_mutex_trylock`. El bloque de inicialización con `PTHREAD_MUTEX_INITIALIZER` ya existía en el código original y se conserva intacto. La corrección preserva exactamente la interfaz esperada y respeta el patrón uniforme que siguen todas las demás funciones de compatibilidad en el mismo archivo.

Comportamiento esperado tras la corrección

Con la corrección aplicada, `pthread_mutex_trylock` delega correctamente en `mthread_mutex_trylock`. El comportamiento esperado del programa de prueba provisto en la orientación es:

- La primera llamada a `pthread_mutex_trylock` retorna 0 (OK) porque el mutex estaba libre.

- La segunda llamada retorna 35 (EDEADLK), ya que `pthread_mutex_trylock` detecta que el mutex ya está tomado por el mismo hilo y devuelve el error correspondiente en lugar de bloquearse.
- Las operaciones `pthread_mutex_unlock` y `pthread_mutex_destroy` completan con éxito.

Para verificar la corrección dentro de la VM bastó recompilar la biblioteca (`cd minix/lib/libmthread && make && make install`), hacer reboot y ejecutar el programa de prueba de la orientación.

2.4 Implementación del comando tree

El comando `tree` fue implementado en `bin/tree/tree.c` e integrado al sistema mediante `bin/tree/Makefile` y la inclusión de `tree` en `bin/Makefile`.

Diseño del algoritmo

Se eligió un enfoque de **recursión explícita** mediante la función `print_tree(path, depth)`, donde `depth` indica el nivel de profundidad actual. Esta elección se justifica porque existe una correspondencia directa entre la profundidad de recursión y el nivel de indentación a imprimir: al entrar en un subdirectorio se incrementa `depth` en uno, y la visualización de cada nivel se obtiene naturalmente a partir de ese contador. Un enfoque iterativo con pila explícita habría requerido almacenar el nivel junto a cada ruta encolada, sin ventaja algorítmica en este caso dado que los árboles de directorios típicamente no son suficientemente profundos como para provocar desbordamiento de pila.

Llamadas al sistema

- `opendir(path)`: abre el directorio indicado y devuelve un descriptor `DIR*` necesario para iterar sus entradas. Retorna `NULL` si el directorio no existe o si no hay permisos suficientes.
- `readdir(dir)`: lee la siguiente entrada del directorio, devolviendo un puntero a `struct dirent` con el nombre del archivo en `d_name`. Retorna `NULL` al agotarse las entradas.
- `closedir(dir)`: libera el descriptor del directorio al terminar la iteración, evitando fuga de descriptores.
- `lstat(path, &st)`: obtiene metadatos del archivo *sin* seguir enlaces simbólicos. Se prefiere `lstat` sobre `stat` precisamente para detectar si la entrada es un enlace simbólico mediante la macro `S_ISLNK` y no descender por él.
- `getcwd(buf, size)`: obtiene la ruta del directorio de trabajo actual cuando no se provee argumento al comando.

Indentación

La profundidad visual se muestra imprimiendo `depth` veces el separador `"| "` (barra vertical más dos espacios), seguido de `"|- "` y el nombre del archivo. A nivel 0 no hay separadores previos, produciendo el efecto estándar de árbol:

```
/ruta/base
|-- directorio_A
|  |-- archivo_1
|  |-- archivo_2
|-- archivo_3
```

Prevención de ciclos

Para evitar seguir enlaces simbólicos — que podrían crear ciclos infinitos o cruzar límites de sistema de archivos no deseados — se usa `lstat` en lugar de `stat`, y se verifica explícitamente con `S_ISLNK` antes de recursar:

Listing 6: Prevención de ciclos mediante `lstat` y `S_ISLNK`

```
1 int check = lstat(fullpath, &entry_stats);
2 if (!(check == -1) && S_ISDIR(entry_stats.st_mode) &&
3     !S_ISLNK(entry_stats.st_mode)) {
4     print_tree(fullpath, depth + 1);
5 }
```

La condición `!S_ISLNK(entry_stats.st_mode)` garantiza que, aunque la entrada sea un directorio alcanzable a través de un enlace simbólico, la recursión no se produce. Esto previene tanto ciclos directos (enlace que apunta a un ancestro en el árbol) como ciclos indirectos.

Código

La implementación completa se encuentra en `bin/tree/tree.c` del repositorio del proyecto: <https://github.com/andr-migue/minix>.

Ejemplos de ejecución

Ruta relativa (`../`) — ejecutado desde `/root`:

```
# tree ../
/
|-- bin
|   |-- cat
|   |-- cp
|   |-- date
|   |-- sh
|   |-- tree
|-- dev
|-- etc
|   |-- motd
|   |-- passwd
|-- home
|-- root
|-- usr
|   |-- bin
|   |-- lib
|   |-- share
```

Comparación con `tree` estándar

El comando `tree` estándar de Linux produce una salida estructuralmente equivalente. Las diferencias son: (1) el `tree` estándar usa caracteres Unicode de caja (+-, \-) mientras que esta implementación usa ASCII (|--), decisión tomada por compatibilidad con la consola de MINIX, que no garantiza soporte Unicode completo; (2) el `tree` estándar muestra al final un resumen de directorios y archivos contados, funcionalidad que queda fuera del alcance mínimo requerido. Los requisitos obligatorios —recursión completa, indentación por nivel, no seguir *symlinks*, y uso exclusivo de `opendir/readdir/lstat`— se cumplen íntegramente.

2.5 Penalización por uso intensivo de CPU

Contexto: arquitectura del scheduler en MINIX

MINIX implementa un planificador en *espacio de usuario*: el kernel no toma decisiones de planificación por sí solo, sino que notifica al servidor `sched` mediante mensajes IPC y este responde indicando qué proceso ejecutar, con qué prioridad y cuánto quantum asignarle. Esta separación es característica de la arquitectura de micronúcleo: un error en el scheduler no colapsa el sistema, y la política de planificación puede modificarse sin tocar el kernel.

El código del scheduler reside en `minix/servers/sched/`. Los archivos centrales son:

- `schedproc.h`: define `struct schedproc`, una tabla con un slot por proceso activo. Almacena toda la información de planificación: prioridad actual, prioridad máxima permitida, quantum asignado, CPU de ejecución y máscara de CPUs.
- `schedule.c`: implementa los manejadores de mensaje:
 - `do_noquantum()`: el kernel envía `SCHEDULING_NO_QUANTUM` cuando un proceso agota su quantum sin bloquearse. Es el punto de observación de comportamiento CPU-intensivo.
 - `do_start_scheduling()`: el kernel pide al scheduler que registre un proceso nuevo. Hay dos variantes: `SCHEDULING_START` (proceso de sistema con prioridad explícita) y `SCHEDULING_INHERIT` (proceso de usuario que hereda del padre).
 - `do_stop_scheduling()`: libera el slot cuando el proceso termina.
 - `do_nice()`: atiende la syscall `nice`, que modifica la prioridad máxima de un proceso.
 - `balance_queues()`: función disparada por alarma cada `BALANCE_TIMEOUT` segundos. Era el único mecanismo de recuperación de prioridad del scheduler original.
 - `schedule_process()`: función interna que comunica al kernel los cambios de prioridad, quantum y CPU mediante `sys_schedule()`.

Convención de prioridades. Las prioridades en MINIX son números enteros *invertidos*: un número menor corresponde a mayor prioridad (la cola 0 es la más prioritaria, reservada al kernel). Para los procesos de usuario:

- `USER_Q = 7`: prioridad base normal de todo proceso de usuario.
- `MIN_USER_Q = 15`: peor prioridad permitida para un proceso de usuario (el número más alto del rango de usuario).

Penalizar un proceso significa *aumentar* su número de cola; recuperarlo significa *disminuirlo*. Esta inversión es crítica para entender todos los cálculos de prioridad que siguen.

Problema: el scheduler original no tenía memoria del comportamiento pasado

El `balance_queues()` original tenía una lógica simple: cada vez que se ejecutaba, subía un nivel de prioridad a cualquier proceso que hubiera sido degradado. No diferenciaba entre procesos de usuario y de sistema, ni tomaba en cuenta con qué frecuencia un proceso había agotado su quantum. Esta ceguera al historial significaba que un proceso CPU-bound (que agota quantaums continuamente) recibía exactamente el mismo tratamiento que uno interactivo que cediera la CPU voluntariamente: ambos eran degradados cuando agotaban su quantum y recuperados por `balance_queues` en la siguiente ventana, sin ninguna penalización diferenciada.

El objetivo de este hito es extender el scheduler para que *observe* el comportamiento de cada proceso dentro de ventanas temporales de 5 segundos y aplique una penalización progresiva a los que demuestren ser CPU-bound, favoreciendo a los procesos interactivos.

Diseño de la política

Elección de la ventana temporal. Se reutilizó el temporizador `BALANCE_TIMEOUT = 5` segundos que ya existía para disparar `balance_queues()`. Esta decisión evita introducir una segunda alarma del sistema (`sys_setalarm`), lo que habría añadido complejidad de sincronización y posibles interacciones con el temporizador existente. Al anclar la ventana de observación al mismo evento de balanceo, la nueva política y la recuperación original quedan naturalmente sincronizadas.

Umbral de clasificación CPU-bound. Se definió `CPU_BOUND_QUANTUM_THRESHOLD = 3`: un proceso que agote 3 o más quantums completos en una ventana de 5 segundos se clasifica como CPU-intensivo. El valor 3 es deliberadamente bajo para reaccionar rápidamente a procesos que consumen CPU de forma sostenida, pero lo suficientemente alto para ignorar ráfagas breves: un proceso que ejecute cálculos intensivos durante 1 segundo y luego se bloquee en I/O no debería ser penalizado, y típicamente no alcanzará 3 quantums completos en una ventana de 5 segundos.

Máximo de penalización. Se definió `MAX_PENALTY_LEVEL = 2`: la prioridad de un proceso CPU-bound puede bajar hasta 2 niveles por debajo de su prioridad base. Este límite tiene dos motivaciones: (1) evitar que un proceso quede tan degradado que no reciba tiempo de CPU en absoluto, lo que podría causar inanición; (2) mantener la recuperación viable en tiempos razonables (2 ventanas de 5 segundos para volver a la base desde el techo de penalización).

Recuperación gradual. Si en una ventana completa un proceso no agota ningún quantum (`quantum_count == 0`) y tiene penalización activa (`penalty_level > 0`), se reduce el nivel en 1. La recuperación es deliberadamente simétrica a la penalización: un nivel por ventana en ambas direcciones. Esto impide que un proceso CPU-bound recupere su prioridad completa gracias a un único momento de inactividad forzada (por ejemplo, esperando un `read` de teclado), lo que haría que la penalización fuera trivialmente evadible.

Zona neutral. Cuando `quantum_count` está entre 1 y `CPU_BOUND_QUANTUM_THRESHOLD - 1` (es decir, 1 o 2 en esta implementación), no se toma ninguna acción sobre la prioridad. Esta zona neutral reconoce que cierta actividad de CPU es normal en cualquier proceso de usuario y no merece castigo. Solo el comportamiento sostenido por encima del umbral activa la penalización.

Cambios en `schedproc.h`: nuevos campos de estado por proceso

Se agregaron tres campos al final de `struct schedproc`:

Listing 7: Campos agregados en `schedproc.h`

```
1 /* CPU-bound process penalty mechanism */
2 unsigned base_priority; /* base priority for gradual recovery */
3 unsigned quantum_count; /* number of full quantums consumed in current
   window */
4 unsigned penalty_level; /* current penalty level (0 = no penalty) */
```

Por qué `base_priority`: sin este campo, no existe forma de calcular cuál debe ser la prioridad actual del proceso en función del nivel de penalización. La prioridad efectiva se define como `base_priority + penalty_level`. Si solo se almacenara `priority`, habría ambigüedad: un proceso con `priority = 9` podría estar en ese valor porque fue penalizado desde `base = 7`, o porque su prioridad base real es 9. `base_priority` resuelve esa ambigüedad y permite aplicar y retirar castigos de forma idempotente.

Por qué `quantum_count`: es la métrica de comportamiento dentro de la ventana actual. Se incrementa en `do_noquantum()` (el punto donde el kernel notifica que el proceso agotó su

quantum) y se reinicia a cero al final de cada ventana en `balance_queues()`. Sin este contador, la política no tendría información sobre qué pasó durante la ventana.

Por qué `penalty_level`: encapsula el castigo acumulado. La prioridad efectiva es siempre `base_priority + penalty_level`, lo que hace que la relación entre el nivel de castigo y la prioridad observable sea directa y auditable. El nivel sube gradualmente (máx. `MAX_PENALTY_LEVEL`) y baja gradualmente, nunca de golpe.

Cambios en `schedule.c`

A. Constantes de configuración.

Listing 8: Constantes de la política de penalización

```
1 /* CPU-bound process penalty mechanism */
2 #define CPU_BOUND_QUANTUM_THRESHOLD 3 /* quantums para penalizar */
3 #define MAX_PENALTY_LEVEL 2 /* niveles max de penalizacion */
```

Definirlas como macros con nombre explícito en lugar de literales numéricos permite ajustar la política con un solo cambio en el código fuente sin afectar la lógica de las funciones. Si se quisiera una ventana más agresiva (por ejemplo, umbral de 2 quantums y penalización de 3 niveles), bastaría con cambiar estas dos líneas.

B. `do_noquantum()`: punto de observación. Esta función se invoca cada vez que el kernel detecta que un proceso agotó su quantum. Es el único punto del código donde se sabe con certeza que el proceso *no* cedió la CPU voluntariamente.

Listing 9: Incremento del contador en `do_noquantum()`

```
1 rmp = &schedproc[proc_nr_n];
2
3 /* Count full quantum consumed by this process */
4 if (!is_system_proc(rmp)) {
5     rmp->quantum_count++;
6 }
7
8 if (rmp->priority < MIN_USER_Q) {
9     rmp->priority += 1; /* lower priority (comportamiento original) */
10 }
```

La guarda `!is_system_proc(rmp)` excluye los procesos de sistema (drivers, servidores como VFS, PM, RS). Los procesos de sistema tienen patrones de ejecución diferentes —a menudo se bloquean esperando mensajes— y sus prioridades están calibradas por el sistema, no por la política de usuario. Penalizarlos podría causar inestabilidad del sistema. La macro `is_system_proc` comprueba si el padre del proceso es el Reincarnation Server (`RS_PROC_NR`), que es quien lanza todos los servicios del sistema.

El bloque original (`priority += 1`) se conserva intacto: sigue degradando el proceso dentro de la ventana actual como siempre lo hizo, además de acumular el conteo para la decisión de fin de ventana.

C. `do_start_scheduling()`: inicialización en tres puntos. Esta función se llama cuando el scheduler toma control de un proceso nuevo. Hay tres puntos de inicialización, cada uno con su razón:

Bloque de inicialización temprana (antes del `switch`):

Los slots de `schedproc[]` son reutilizados cuando un proceso termina y uno nuevo ocupa su posición. Sin inicialización explícita, los nuevos campos contendrían basura del proceso anterior,

haciendo que el proceso nuevo herede penalización o conteos que no le corresponden. Este bloque garantiza un estado limpio independientemente del contenido previo del slot.

Rama SCHEDULING_START (procesos de sistema con prioridad explícita):

Listing 10: Inicialización en rama SCHEDULING_START

```
1 case SCHEDULING_START:
2     rmp->priority      = rmp->max_priority;
3     rmp->base_priority = rmp->max_priority; /* NUEVO */
4     rmp->time_slice    = m_ptr->m_lsys_sched_scheduling_start.quantum;
5     rmp->quantum_count = 0;                /* NUEVO */
6     rmp->penalty_level = 0;                /* NUEVO */
7     break;
```

Para procesos de sistema, `base_priority` debe reflejar su prioridad real asignada (`max_priority`), no el valor genérico `USER_Q`. Si se usara `USER_Q` como base para un proceso de sistema con prioridad 4, los cálculos de penalización serían incorrectos. Aunque los procesos de sistema no son penalizados (`is_system_proc` los excluye), mantener `base_priority` coherente evita confusión si en el futuro se revisara esa exclusión.

Rama SCHEDULING_INHERIT (procesos de usuario que heredan del padre):

Listing 11: Inicialización en rama SCHEDULING_INHERIT

```
1 case SCHEDULING_INHERIT:
2     rmp->priority      = schedproc[parent_nr_n].priority;
3     rmp->base_priority = schedproc[parent_nr_n].priority; /* NUEVO */
4     rmp->time_slice    = schedproc[parent_nr_n].time_slice;
5     rmp->quantum_count = 0;                /* NUEVO */
6     rmp->penalty_level = 0;                /* NUEVO */
7     break;
```

El hijo hereda la prioridad *actual* del padre como punto de partida. Se usa `priority` (no `base_priority`) del padre porque es la prioridad real con la que el hijo empieza a ejecutarse. El hijo parte sin penalización (`penalty_level = 0`) ni historial (`quantum_count = 0`): cada proceso construye su reputación de forma independiente.

D. `balance_queues()`: el corazón de la política. Esta es la función más significativamente modificada. Se ejecuta cada 5 segundos y aplica la política completa. La implementación se encuentra en `minix/servers/sched/schedule.c` del repositorio: <https://github.com/andr-migue/minix>.

Decisiones de implementación relevantes:

- **Clamp en penalización** (`new_priority > MIN_USER_Q`): las prioridades en MINIX son números invertidos. `MIN_USER_Q` es el número más alto del rango de usuario; valores mayores corresponden a colas de sistema. Si `base_priority + penalty_level` superara `MIN_USER_Q`, el proceso entraría en rango de sistema con consecuencias imprevisibles. El clamp garantiza que ningún proceso de usuario pueda ser empujado fuera de su rango válido. En código fuente la condición correcta es `if (new_priority > MIN_USER_Q)` —como `MIN_USER_Q` es el número más alto del rango, sobrepasar ese límite significa un número aún mayor, de ahí la comparación con `>`. *Nota:* durante las pruebas se detectó que la versión inicial tenía la condición invertida (`<` en lugar de `>`), lo que impedía que el clamp se disparara. Esto fue corregido antes de la validación experimental.
- **Clamp en recuperación** (`new_priority > rmp->max_priority`): simétricamente, al recuperar prioridad hay que asegurarse de no superar la prioridad máxima autorizada del proceso (`max_priority`). Un proceso cuya prioridad máxima fue reducida por `nice` no debería recuperarse hasta `USER_Q` si ya no tiene ese permiso.

- **Condición `tmp->priority != new_priority`:** solo se llama a `schedule_process_local()` si la prioridad efectivamente cambia. Esta optimización evita enviar mensajes IPC innecesarios al kernel cuando el proceso ya está en la prioridad correcta (p. ej., un proceso en techo de penalización que sigue siendo CPU-bound: ya está en `MIN_USER_Q` y no hay que reschedularlo).
- **Reinicio de `quantum_count = 0` siempre:** el reinicio ocurre independientemente del resultado de la evaluación. Si estuviera dentro del `if/else if`, los procesos en la zona neutral (`quantum_count` de 1 o 2) acumularían conteos entre ventanas, corrompiendo la semántica de “cuantos agotados en la ventana actual”.

Flujo completo del mecanismo

El ciclo de vida de la penalización para un proceso CPU-bound es el siguiente:

1. El proceso se crea y `do_start_scheduling()` inicializa: `priority = base_priority = USER_Q = 7`, `quantum_count = 0`, `penalty_level = 0`.
2. El proceso ejecuta código intensivo en CPU. Cada vez que agota su quantum, el kernel envía `SCHEDULING_NO_QUANTUM` y `do_noquantum()` incrementa `quantum_count`.
3. Tras 5 segundos, `balance_queues()` se ejecuta. Si `quantum_count >= 3`: `penalty_level` sube a 1, `priority` pasa a 8.
4. En la siguiente ventana, si el proceso sigue siendo CPU-bound: `penalty_level` sube a 2, `priority` pasa a 9. Ya alcanzó el techo.
5. Mientras siga siendo CPU-bound, la prioridad se mantiene en 9 (`penalty_level` no puede superar `MAX_PENALTY_LEVEL = 2`).
6. Si el proceso cambia de comportamiento (p. ej., empieza a hacer I/O) y no agota ningún quantum en una ventana: `penalty_level` baja a 1, `priority` vuelve a 8.
7. Una ventana más sin agotar quants: `penalty_level` baja a 0, `priority` vuelve a 7. Recuperación completa.

Validación experimental

Para verificar el comportamiento del mecanismo dentro de la VM se implementaron dos programas de prueba en `sched_test/`:

- `cpu_bound.c`: ejecuta un bucle aritmético infinito (`while(1) x++`) sin ceder la CPU voluntariamente. Agota su quantum en cada ciclo de planificación, comportamiento que el scheduler debe detectar y penalizar progresivamente.
- `interactive.c`: duerme un segundo entre iteraciones (`sleep(1)`), cediendo la CPU voluntariamente. Nunca agota un quantum completo, por lo que no debe recibir ninguna penalización.

Ambos programas se compilaron y ejecutaron en paralelo dentro de la VM:

Listing 12: Compilación y ejecución de los programas de prueba

```
cd sched_test
make
./cpu_bound & ./interactive & top
```

La validación se realizó observando las prioridades (`PRI`) de ambos procesos con `top` durante varias ventanas de 5 segundos. Las figuras siguientes muestran la evolución del sistema durante la prueba.

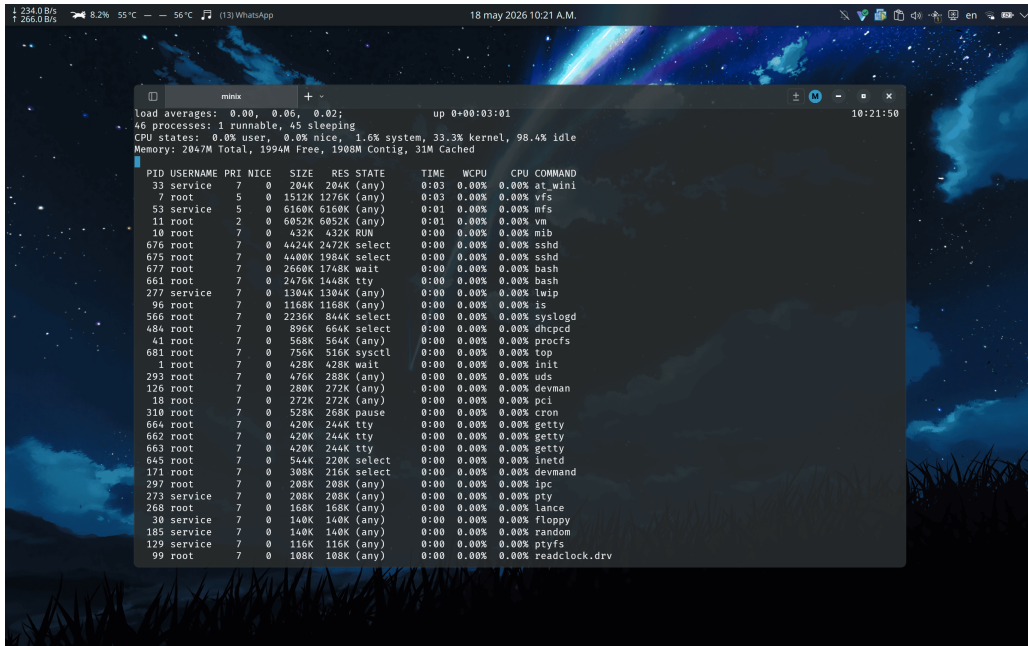


Figure 1: Estado del sistema con el scheduler original del repositorio, antes de aplicar ninguna modificación. La CPU se encuentra al 98.4% idle, todos los procesos operan en prioridad base (PRI=7) y ningún proceso consume tiempo de CPU apreciable.

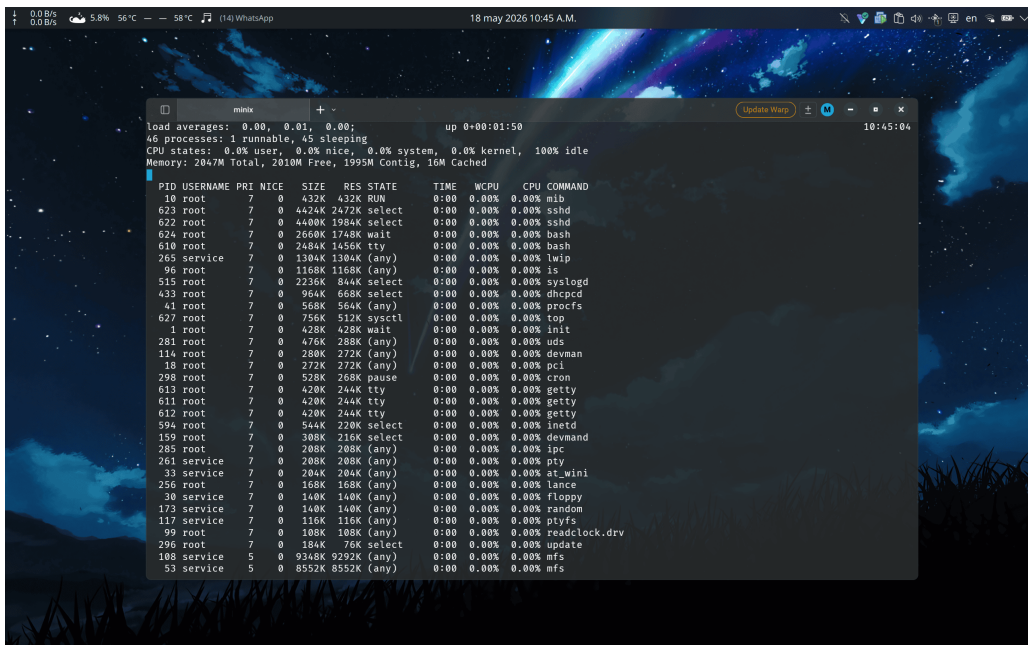


Figure 2: Sistema con el scheduler modificado pero sin los programas de prueba en ejecución. La CPU se encuentra al 100% idle y todos los procesos permanecen en prioridad base (PRI=7), confirmando que el mecanismo no introduce ninguna perturbación en ausencia de procesos CPU-bound.

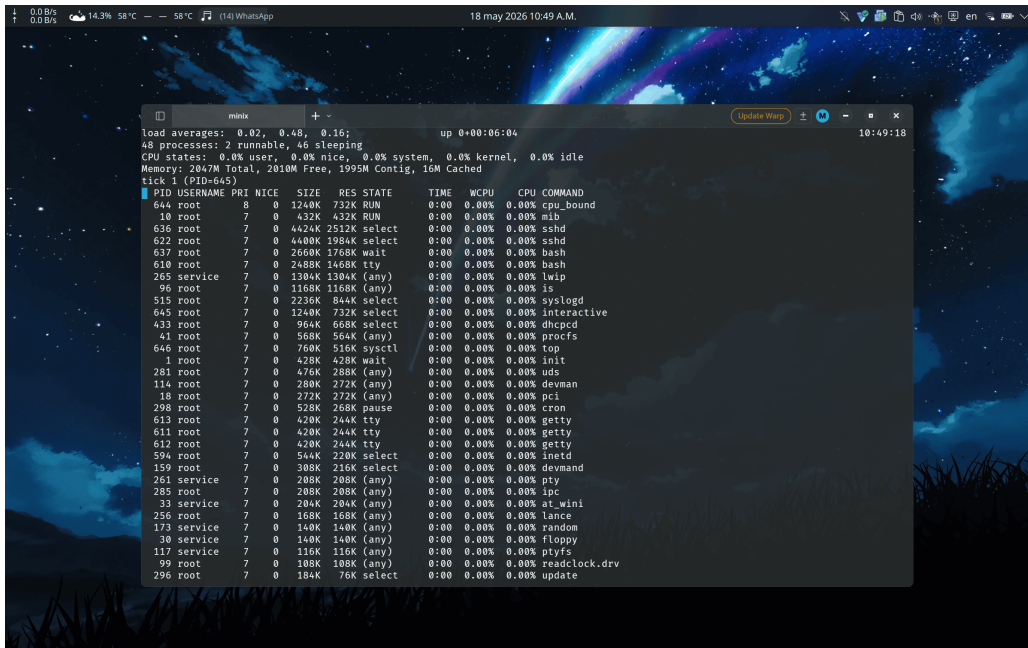


Figure 3: Inicio de los tests de validación y primera penalización aplicada tras la primera ventana de `BALANCE_TIMEOUT`: `cpu_bound` (PID 644) sube a `PRI=8` al haber agotado `quantums` de forma sostenida. El proceso `interactive` permanece en prioridad base (`PRI=7`) sin penalización.

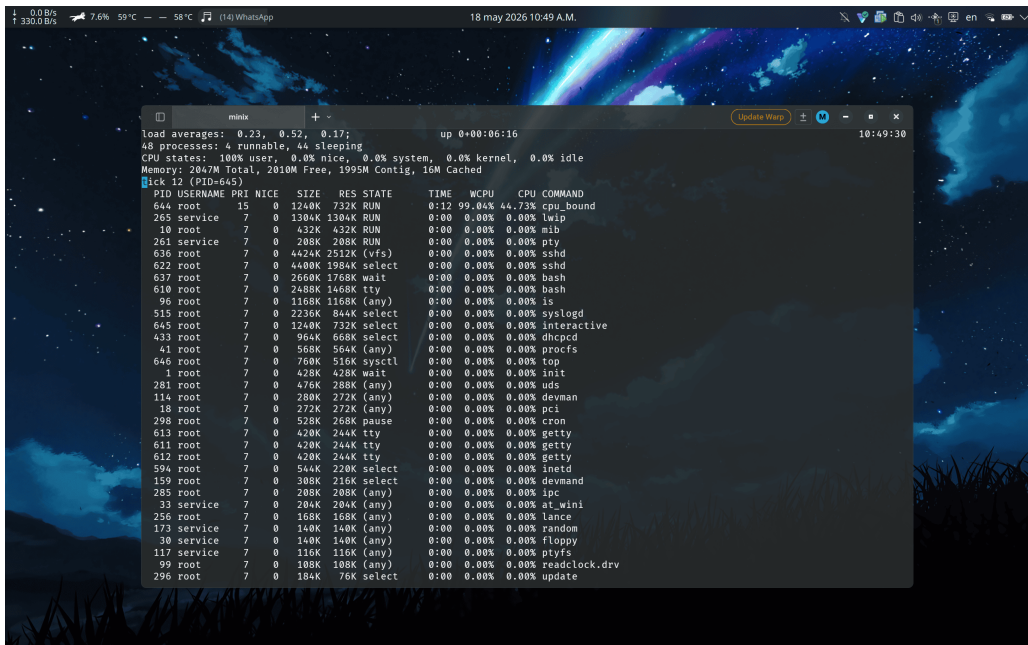


Figure 4: Penalización máxima alcanzada 5 segundos después: `cpu_bound` (PID 644) escala a `PRI=15` (`MIN_USER_Q`, techo del rango de usuario), acaparando el 44.73% de CPU (`WCPU=89.44%`) con la peor prioridad posible. La carga de usuario sube al 100%. El proceso `interactive` sigue sin penalización (`PRI=7`), confirmando que el mecanismo discrimina correctamente entre ambos tipos de proceso.

3 Resultados globales y discusión

Los cuatro componentes del proyecto fueron implementados y funcionan correctamente según los criterios de aceptación establecidos en la orientación:

- **Personalización de `/etc/motd`:** *completa*. El mensaje personalizado se muestra correctamente al iniciar sesión, con referencia explícita a la Facultad de Matemática y Computación de la UH.
- **Corrección de `pthread_mutex_trylock`:** *completa*. La causa raíz (recursión directa en lugar de delegación a `pthread_mutex_trylock`) quedó eliminada con una modificación de una sola línea. El comportamiento resultante es correcto según la semántica de la función: la segunda llamada debe retornar `EDEADLK` en lugar de bloquearse indefinidamente.
- **Comando `tree`:** *completo*. La herramienta muestra la estructura de directorios recursivamente, respeta la indentación por nivel, no sigue enlaces simbólicos y maneja correctamente errores de permiso.
- **Penalización CPU-bound en el scheduler:** *completa y validada experimentalmente*. Los campos de estado por proceso (`base_priority`, `quantum_count`, `penalty_level`) fueron añadidos a `schedproc.h` y la lógica de penalización y recuperación gradual fue implementada en `do_noquantum()` y `balance_queues()`. La validación mediante los programas `cpu_bound` e `interactive` confirmó el comportamiento predicho: el proceso CPU-intensivo fue penalizado hasta `MIN_USER_Q` en ventanas sucesivas de 5 segundos, mientras el proceso interactivo mantuvo su prioridad base sin cambios (véase figuras 3 y 4). Durante las pruebas se detectó y corrigió un bug en la condición de clamp de `balance_queues()` (< en lugar de >), que impedía que el límite de penalización se aplicara correctamente.

La principal dificultad técnica encontrada fue comprender el flujo de mensajes entre el kernel y el servidor `sched`: el kernel no llama directamente a funciones del scheduler, sino que envía un mensaje de tipo `SCHEDULING_NO_QUANTUM` cuando un proceso agota su quantum. Solo al entender este mecanismo fue posible identificar `do_noquantum()` como el punto correcto para incrementar el contador. Una confusión inicial llevó a considerar modificar directamente `minix/kernel/proc.c`, lo que habría sido incorrecto dado que el objetivo era modificar el planificador de espacio de usuario.

En cuanto al bug de `pthread`, la causa (recursión directa) resultó evidente una vez localizado el archivo correcto. El proceso de búsqueda fue instructivo porque requirió seguir la cadena `pthread_mutex_trylock` → `pthread_compat.c` → `pthread_mutex_trylock` → `mutex.c`, lo que ilustra la arquitectura de capas de la biblioteca de hilos de MINIX y la importancia de distinguir la capa de compatibilidad de la implementación real.

Una lección general del proyecto es que la arquitectura de micronúcleo de MINIX facilita considerablemente la experimentación: errores en el scheduler no colapsan el sistema completo, y los cambios pueden revertirse sin consecuencias irreversibles, a diferencia de modificar un kernel monolítico.

4 Conclusiones

El proyecto cumplió íntegramente con los objetivos planteados. Cada uno de los cuatro puntos de trabajo abordó un aspecto diferente y complementario de los sistemas operativos: la modificación de `motd` introdujo el flujo básico de edición-verificación en MINIX; el bug de `pthread` puso en práctica técnicas de depuración en código de biblioteca de sistema; el comando `tree` desarrolló la capacidad de interactuar con el sistema de archivos mediante llamadas al sistema; y la extensión

del scheduler requirió comprender y modificar la política de planificación de un sistema real con impacto observable en el comportamiento de los procesos.

Desde el punto de vista formativo, el proyecto demuestra que modificar un sistema operativo real —incluso uno de diseño académico— exige comprender no solo el código directamente involucrado, sino también los mecanismos de comunicación entre componentes (IPC por mensajes en MINIX), las convenciones de compilación del sistema y la relación entre las capas de abstracción.

Como mejoras posibles al trabajo realizado se proponen: (1) extender el scheduler para distinguir más de dos clases de procesos, con políticas de penalización diferenciadas por tipo; (2) exponer estadísticas de planificación por proceso a través de `/proc` para facilitar la validación experimental sin necesidad de `top`; y (3) añadir al comando `tree` el conteo de archivos y directorios al final de la ejecución, llevándolo a paridad funcional con la implementación estándar de Linux.

5 Referencias consultadas

1. Tanenbaum, A. S. & Woodhull, A. S. (2006). *Operating Systems: Design and Implementation* (3rd ed.). Prentice Hall.
2. MINIX 3 Official Website. Recuperado de <https://www.minix3.org>
3. MINIX 3 Wiki — Developer’s Guide. <https://wiki.minix3.org/doku.php?id=developersguide:start>
4. Documentos de orientación del proyecto:
 - Orientación del proyecto.pdf
 - Cronograma de hitos.pdf
 - THE MATCOM MINIX_Informe técnico.pdf

6 Declaración de uso de IA

Se utilizó IA generativa (Claude, Anthropic) como apoyo en las siguientes partes del proyecto:

- **Redacción del informe técnico:** estructuración de secciones, revisión de redacción técnica y generación de la versión $\text{\LaTeX}$ del documento.
- **Síntesis de requisitos:** extracción y organización de los criterios de aceptación a partir de los documentos PDF de orientación.
- **Documentación de conceptos teóricos:** consultas sobre principios de planificación de procesos, IPC en micronúcleos y el modelo de capas de la biblioteca de hilos de MINIX.
- **Implementación de tree:** búsqueda de las funciones estándar de C para manejo de directorios y archivos (`opendir`, `readdir`, `closedir`, `lstat`) y su uso correcto en el contexto de MINIX.